



CSC241 Object Oriented Programming (Java)
Theory Assignment 1:
Object Oriented Analysis of a Smart Campus Allocation System

Name: Aleeha Khurram
Registration No. : SP26-BAI-009
Instructor: Muhammad Shahid Bhatti

My Extra Condition:

Resource limitation (my registration number ends in 9). Some equipment is movable and scarce, so room and equipment need separate allocation decisions. I include this in every task where it matters.

Task 1: Stakeholder and Requirement Analysis

In Task 1, I am listing all the stakeholders; who use the system, explain where their needs clash with each other, and then separate what the whole scenario tells us from what I had to assume.

Listing Stakeholders and Their Needs:

Stakeholder	One need
Students	Fair access to rooms and to scarce equipment such as projectors.
Faculty members	Priority for scheduled lectures, including the equipment needed for the class.
Administration	High use of rooms and equipment so that nothing sits empty.
Security staff	Traceable access records that show who used which space and when.
Maintenance staff	A way to block an unsafe room (or faulty equipment) immediately.
Disability support staff	Accessible rooms and equipment stay available for people who need them, without the system storing anyone's disability.

Three Genuine Requirement Conflicts:

Conflict	Reason	My Assumptions and Decisions
1. Fairness vs. Faculty Priority	Students want equal access, but faculty needs priority for teaching. It is worse for equipment, because there are only a few projectors.	I assume "fair" means each student can hold at most 2 to 3 active bookings per week. Faculty priority applies only to scheduled lectures or urgent academic activity. A displaced student is notified and offered another slot another time.
2. High utilization vs. Safety	The administration wants rooms and equipment to be used as much as possible, but maintenance must be able to block an unsafe room at once.	Safety is the top priority. A block by maintenance staff implies immediately, and affected bookings are rescheduled or cancelled even if utilization drops.
3. Privacy vs. Traceability	Security needs a record of who used a space, but users expect their personal information to stay private.	Audit records store all the activity, but only security and admin roles can read them. Ordinary users only see the status: "available" or "occupied".

Difference Between Facts and Assumptions:

Facts (given in scenario)	Assumptions (made by me)
There are four kinds of requests, each with different capacity, equipment and approval rules.	Fair access means at most 2 to 3 active bookings per week, per student.
A person can hold several roles, then roles and permissions can change during the semester.	A repeated or frequent booking is limited to 8 occurrences, and each occurrence is treated as its own reservation.
Due to emergencies, confirmed reservations can be relocated or cancelled. The system must keep the audit information and notify people.	For simultaneous requests, the earliest one wins, and a tie of time is decided by the priority of the stakeholder's role.

Facts (given in scenario)	Assumptions (made by me)
Equipment is movable and scarce, so room and equipment may need separate decisions (my given condition).	An equipment item is in only one place at a time and is booked separately from the room.
The meaning of fair access, recurring limits and simultaneous requests has not been decided.	Reservations are not completely deleted from the record. Only their state changes, so the audit trail stays complete.

Task 2: Object Oriented Conceptual Model

Classes and their responsibilities:

I have selected 8 classes. Each one has a single main job.

Class	Responsibility	Attributes / operations
User	A person who uses the system, i.e student, teacher, or staff.	Has: name, contact info. Can: get a new role, check if allowed to do something.
UserRole	The role a person currently has, like student or teaching assistant.	Has: role type, end date. Can: check if the role is still active.
Room	A place you can book, like a classroom or a lab. Also knows if it's blocked.	Has: seat count, current status. Can: check if free, hold a time slot, block itself.
Equipment	Any movable item, like a projector, that is limited in number.	Has: status, current location. Can: hold a time slot, move to another room.
Booking	One request to use a room at a certain time.	Has: type of request, time slot, current stage. Can: confirm, cancel, move to a new room.
EquipmentBooking	The separate decision about the booking of equipment.	Has: time slot, approved or not. Can: approve or deny.
ApprovalRule	The rule that decides if a booking should be allowed or not.	Can: check a booking against the rule.
Log	Keeps a record of what happened, and sends a message about it.	Has: message, time. Can: save a record, send a message.

UML class diagram:

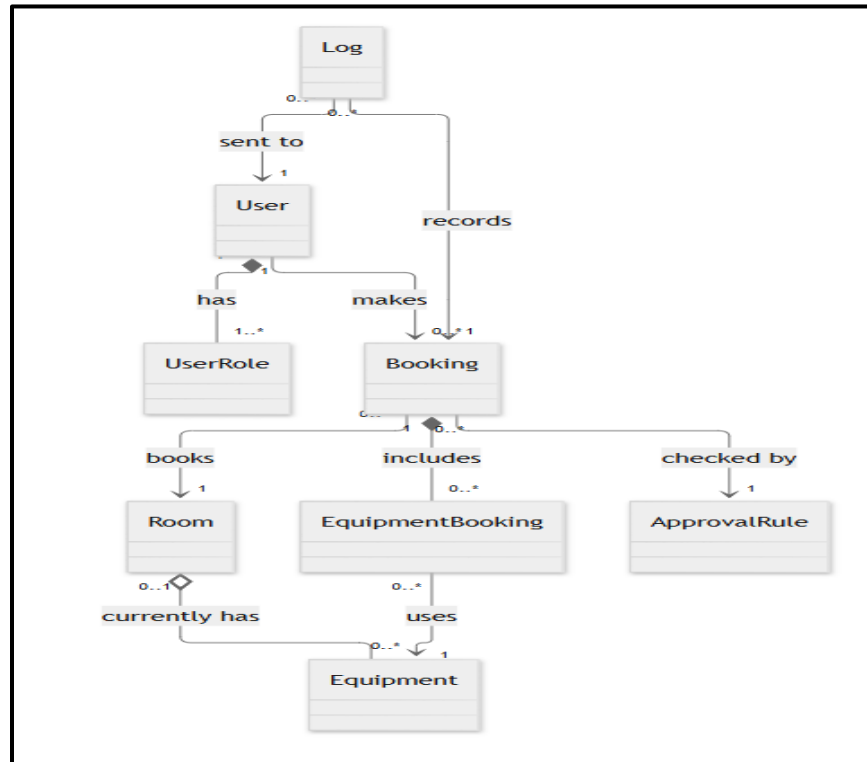


Figure 01: UML Class Diagram of Smart Campus Allocation System

UML Diagram Description:

An arrow shows that one class is connected to another. The numbers next to each arrow show how many objects can be linked on each side. A filled diamond means the small part cannot exist without the big part. A hollow diamond means the small part can still exist on its own.

Separation of Responsibilities:

Any single class does not control everything. Room, Equipment, and Booking each look after their own information. ApprovalRule only holds the rule, and Log only saves records and sends messages. Booking connects everything together by checking the rule, booking the room, and booking the equipment, but it does not control the other classes directly. This way, changing one part of the system does not affect the rest.

Task 3: Abstraction, Encapsulation and Invariants

The three classes I chose: Booking, Room, and Equipment.

What each class abstracts, what is hidden, and how it can be changed:

Class	What it abstracts (keeps / ignores)	Hidden information	Can only be changed through
Booking	It keeps the important facts about a request: who asked, what time, and what stage it is at. It ignores the small details, like why the person wants the room.	current stage	confirm(), cancel(), move to a new room
Room	It keeps the facts needed for booking: how many people fit, and whether it is free or blocked. It ignores things like furniture or wall colour.	current status	hold a time slot, block itself
Equipment	It keeps what matters for booking: its status and where it currently is. It ignores things like brand or price.	current location	hold a time slot, move to another room

Four invariants (rules that must always stay true):

1. Two confirmed bookings can never use the same room at the same time, which prevents the double booking of a single room.
2. One piece of equipment can never be booked for two overlapping times. It means that an equipment can't be reserved for two places simultaneously.
3. A blocked room cannot accept a new confirmed booking, as it is important with respect to the safety concerns.
4. Once a booking is cancelled, it can never become confirmed again. Which means that a new booking would be needed, the cancelled one wouldn't be possible to retrieve.

How encapsulation stops invalid states:

The important information inside Room, Equipment, and Booking is hidden, and it can only be changed through their own actions, like hold() or confirm(). Because no other class can change this information directly, no one can accidentally create a situation that breaks the rules above, like two bookings overlapping in the same room.

Task 4: Alternative Designs and Decision

Two ways to represent a user with changing roles:

- **Option A, Inheritance:** Make separate classes like Student, Teacher, and Staff, each one built from User.
- **Option B, Composition (UserRole):** Keep one User class, and give it one or more UserRole objects.
-

Comparison:

Criteria	Option A: Inheritance	Option B: Composition (UserRole)
Multiple roles at once	Hard. A person who is both a student and a teaching assistant needs a brand-new mixed class.	Easy. The user just gets two UserRole objects.
Changing roles mid-semester	Hard. The person's whole class would need to change.	Easy. Just add or remove a UserRole.
Duplicate code	The same permission checks get repeated in each subclass.	Written once, inside UserRole.
Extensibility	Every new role type needs a whole new class.	A new role is just new data, no new class needed.

My choice and its Explanation:

I chose Option B, composition with UserRole. The scenario says people can hold more than one role, and roles can change during the semester. Option B lets a user gain or lose a role easily, without rebuilding the whole User object, so it fits the requirements better.

Task 5: Behaviour and Edge Cases

Life cycle of a Booking (state table):

From	What happens	To	From
Requested	Passes the basic checks	Pending	Requested
Pending	ApprovalRule says yes, and Room holds the slot	Confirmed	Pending
Pending	ApprovalRule says no, or the room is taken	Rejected	Pending
Requested / Pending	User cancels it	Cancelled	Requested / Pending
Confirmed	Time starts	In Use	Confirmed
In Use	Time ends	Completed	In Use
Confirmed	Room is closed, another room is found	Moved	Confirmed
Confirmed	Room is closed, no other room found	Cancelled	Confirmed
From	What happens	To	From

The three cases

- Two people request the same room at the same time.**
Room checks both requests. The one that arrived first gets Confirmed. The other one gets Rejected and is shown other free rooms.
- A maintenance block happens after a booking is confirmed.**
Room blocks itself right away. Any booking for that room tries to move to another room. If no room is free, the booking is cancelled. If equipment was booked too, it moves with the booking.

3. **An emergency needs to cancel or move many bookings at once.**

All rooms in the area block themselves. Each booking checks itself and tries to move to a new room, or gets cancelled if none is free. Log sends a message to every person affected, and saves a record of each change.

Task 6: Independent Exploration

Immutable Objects:

An **Immutable Object** is an object that cannot be changed after it is created. If we need to update any of its information, we can't edit the existing object. For this purpose, we need to make a new object in order to replace the existing one. [[1]]

It is a helpful approach as it stops other parts of the program from accidentally changing the data while somewhere else is using it. [[1],[2]].

For instance, in our Smart Campus Allocation System, we can use this concept in **Booking** for a time slot inside it. When a reservation gets rescheduled, then instead of changing the start and end times of the previous booking, we throw away the old time slot and simply replace it by creating a new one. It keeps our data completely safe and correct, and avoids any sort of confusion as the need for any extra code to double-check can be eliminated easily. [[2]]

References

- [1] Baeldung, "Immutable Objects in Java," *Baeldung*. [Online]. Available: <https://www.baeldung.com/java-immutable-object>
- [2] J. Bloch, *Effective Java*, 3rd ed. Boston, MA, USA: Addison-Wesley, 2018, item 17.

Generative AI Disclosure

I used Claude by Anthropic, to help me understand the requirements of this assignment and to learn the terminology and concepts behind object oriented design, such as composition, aggregation, encapsulation, and invariants. I also used it to help design the layout of this Word document, and to help me find authentic, credible sources to cite in Task 6. The UML class diagram was created by me using mermaid.live, based on the classes, responsibilities, and relationships I had already worked out in Task 2. I acknowledge that I have used generative AI to help me understand the base concepts and the requirements of the assignment, then I used my own reasoning and understanding to write down all the decisions and explanations required.